

**SOFTWARE DESIGN DOCUMENT PARALLEL PASSWORD HASHING
MENGUNAKAN CPU DAN GPU (CUDA)**



**DOSEN PENGAMPU :
SEPTIAN GEGES, S.KOM., M.KOM.**

**DISUSUN OLEH :
KELOMPOK 1**

KELVIN ANDRE HARDIAN	(2430305030011)
MUHAMMAD SHAUFI WAFA B.S	(2430205030028)
GIOVANNI MARANATHA	(2430305030022)
FAJAR OKTAVIAN RAMADHAN	(2430205030023)

**JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PALANGKA RAYA**

2026

1. Pendahuluan

Perkembangan komputasi paralel memungkinkan penyelesaian masalah kompleks dengan lebih cepat dibandingkan metode sekuensial. Salah satu kasus yang relevan adalah brute-force password hashing, di mana sistem mencoba berbagai kemungkinan password untuk menemukan kecocokan dengan hash tertentu. Metode ini memiliki kompleksitas tinggi sehingga membutuhkan optimasi melalui pemanfaatan CPU dan GPU.

Rumusan Masalah

- 1) Bagaimana cara mengimplementasikan brute-force hashing?
- 2) Bagaimana perbandingan performa CPU dan GPU?

Tujuan

- 1) Membangun sistem brute-force hashing
- 2) Mengimplementasikan paralelisme dengan GPU
- 3) Membandingkan performa CPU vs GPU

2. Dasar Teori

A. Hashing

Hashing adalah proses mengubah data menjadi nilai tetap menggunakan algoritma seperti MD5.

B. Brute Force

Metode mencoba semua kemungkinan hingga menemukan hasil yang cocok.

C. Parallel Computing

Teknik menjalankan banyak proses secara bersamaan untuk mempercepat komputasi.

D. CUDA & Numba

CUDA digunakan untuk GPU computing, sedangkan Numba memungkinkan Python menjalankan kode CUDA.

3. Desain Sistem (SDD)

A. Arsitektur Sistem

Sistem terdiri dari beberapa modul:

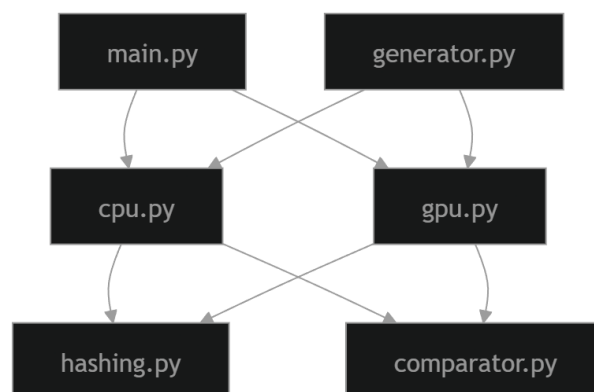
- main.py → kontrol utama
- cpu.py → brute force CPU
- gpu.py → brute force GPU (CUDA)
- hashing.py → fungsi hashing
- generator.py → generator password
- comparator.py → pembandingan hash

B. Alur Sistem

1. Input hash
2. Generate kandidat password
3. Hash setiap kandidat
4. Bandingkan dengan target
5. Jika cocok → output password

4. Diagram Arsitektur

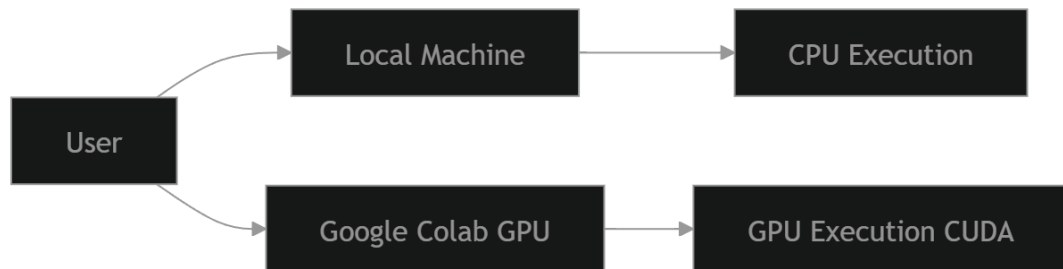
A. Component Diagram



Gambar 1 Component Diagram

Diagram ini menunjukkan bahwa main.py bertindak sebagai kontrol utama yang mengatur eksekusi CPU atau GPU. Modul cpu.py dan gpu.py menggunakan hashing.py untuk proses hashing, serta menerima input kandidat dari generator.py. Hasil hashing dibandingkan menggunakan comparator.py.

B. Deployment Diagram

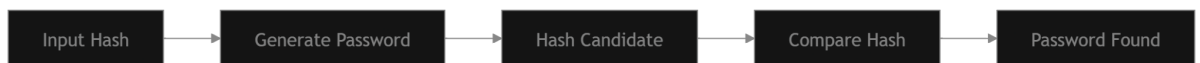


Gambar 2 Deployment Diagram

Diagram ini menunjukkan distribusi sistem di dua environment:

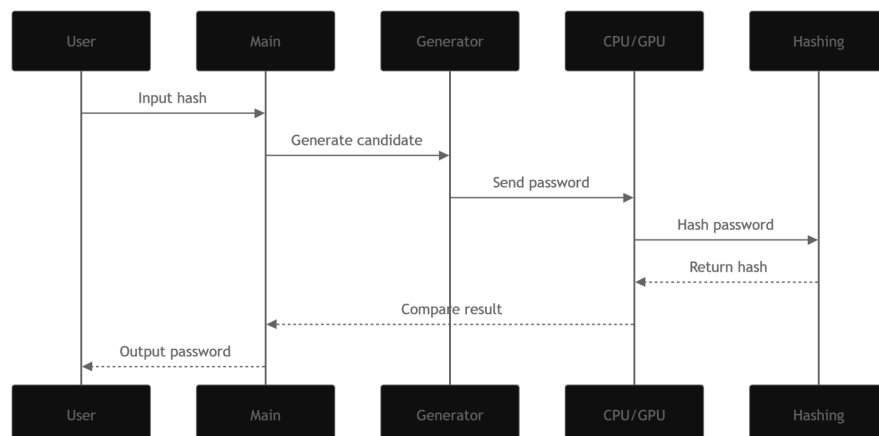
- Local Machine → menjalankan CPU version
- Google Colab → menjalankan GPU version dengan CUDA

C. Data Flow Diagram



Gambar 3 Data Flow Diagram

D. Sequence Diagram



Gambar 4 Sequence Diagram

5. Implementasi

Teknologi

1. Python
2. Numpy
3. Numba
4. CUDA
5. Google Colab

CPU Implementation

Brute-force dilakukan secara sekuensial menggunakan loop.

GPU Implementation

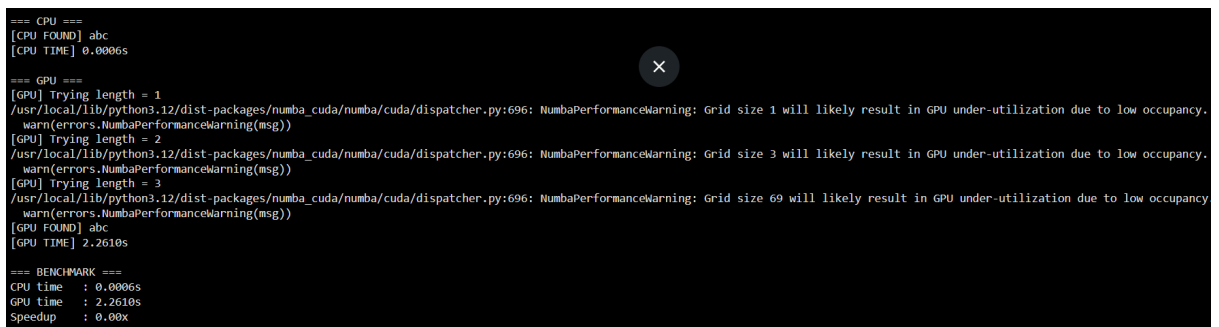
Menggunakan CUDA kernel dengan Numba untuk menjalankan banyak thread secara paralel.

6. Pengujian

Environment

1. Local (CPU)
2. Google Colab (GPU T4)

Hasil



```
=== CPU ===
[CPU FOUND] abc
[CPU TIME] 0.0006s

=== GPU ===
[GPU] Trying length = 1
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
[GPU] Trying length = 2
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 3 will likely result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
[GPU] Trying length = 3
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 69 will likely result in GPU under-utilization due to low occupancy
  warn(errors.NumbaPerformanceWarning(msg))
[GPU FOUND] abc
[GPU TIME] 2.2610s

=== BENCHMARK ===
CPU time   : 0.0006s
GPU time   : 2.2610s
Speedup    : 0.00x
```

Gambar 5 Hasil Pengujian dengan dataset kecil

```

... == CPU ==
[CPU FOUND] fajarr
[CPU TIME] 1713.1189s

== GPU ==
[GPU] Trying length = 1
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely re
warn(errors.NumbaPerformanceWarning(msg))
[GPU] Trying length = 2
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 3 will likely re
warn(errors.NumbaPerformanceWarning(msg))
[GPU] Trying length = 3
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 69 will likely r
warn(errors.NumbaPerformanceWarning(msg))
[GPU] Trying length = 4
[GPU] Trying length = 5
[GPU] Trying length = 6
[GPU] Trying length = 7
[GPU] Not found

== BENCHMARK ==
CPU time : 1713.1189s
GPU time : 6.3170s
Speedup : 271.19x

```

Gambar 6 Hasil Pengujian dengan dataset besar

1. CPU: stabil tapi lambat untuk dataset besar
2. GPU: lebih cepat untuk banyak kombinasi

Contoh Output

1. Input hash: 900150983cd24fb0d6963f7d28e17f72
2. Output: abc

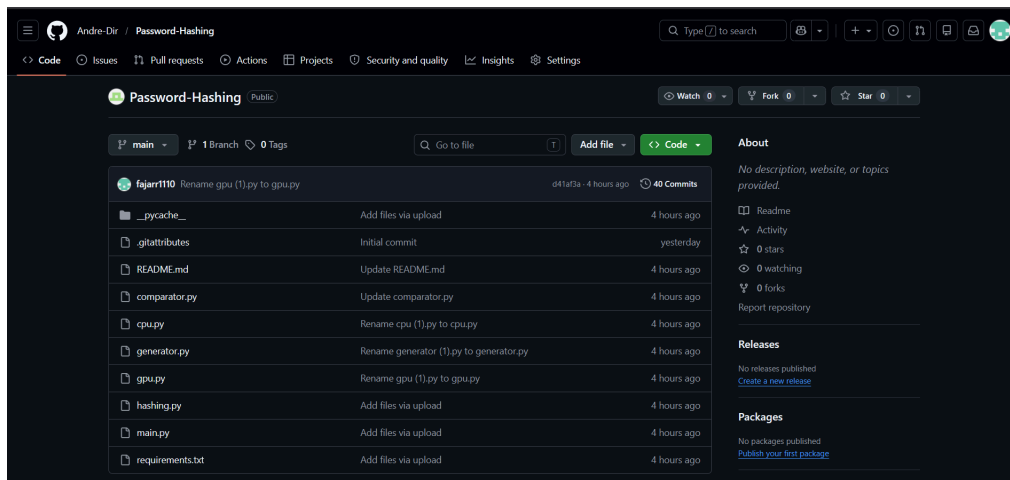
7. Analisis

1. GPU unggul dalam skala besar
2. CPU cocok untuk testing kecil
3. CUDA butuh konfigurasi khusus
4. Overhead GPU terlihat pada dataset kecil

8. Kesimpulan

Proyek ini berhasil mengimplementasikan sistem brute-force password hashing dengan pendekatan CPU dan GPU. Hasil menunjukkan bahwa GPU memberikan performa yang lebih baik untuk skala besar, sedangkan CPU lebih sederhana dalam implementasi. Penggunaan CUDA melalui Numba memberikan fleksibilitas dalam pengembangan sistem paralel berbasis Python.

9. Lampiran



Gambar 7 Struktur Repository

Link repository : <https://github.com/Andre-Dir/Password-Hashing>

10. Risiko & Mitigasi

Risiko yang Diprediksi :

A. Bottleneck pada CPU

- CPU bekerja secara sekuensial sehingga lambat untuk dataset besar
- Terjadi keterbatasan performa saat jumlah kombinasi password meningkat

B. Underutilization GPU

- Grid size kecil menyebabkan GPU tidak optimal
- Tidak semua thread terpakai secara maksimal

C. Dependency CUDA & Environment

- CUDA Toolkit tidak terdeteksi
- Error seperti nvvm.dll not found
- Ketergantungan pada versi Python (≤ 3.11)

D. Single Point of Failure

- Jika main.py error → seluruh sistem gagal
- Jika GPU gagal → tidak bisa jalankan paralel

E. Masalah Environment Lokal

- Drive C penuh → instalasi gagal
- Konflik PATH Python (multi version)

Mitigasi :

1. **Gunakan GPU untuk workload besar**
 - a. CPU hanya untuk testing kecil
 - b. GPU untuk brute-force skala besar
2. **Optimasi CUDA Kernel**
 - a. Gunakan grid dan block yang lebih besar
 - b. Hindari grid size = 1
3. **Gunakan Google Colab sebagai fallback**
 - a. Jika CUDA lokal gagal → pakai Colab (T4 GPU)
4. **Pisahkan modul**
 - a. Modular design → error tidak merusak seluruh sistem
5. **Gunakan Python versi stabil**
 - a. Disarankan Python 3.10 / 3.11

11.Task Matrix (Manajemen Tim)

Task matrix digunakan untuk membagi pekerjaan antar anggota tim menggunakan Github.

Tugas	Penanggung Jawab
Desain Arsitektur	Muhammad Shaufi Wafa B.S
Implementasi CPU	Fajar Oktavian Ramadhan
Implementasi GPU	Kelvin Andre Hardian
Testing & Evaluasi	Giovanni Marantha
Dokumentasi (SDD)	Semua anggota